Solving the XOR problem

Posted on April 02, 2021

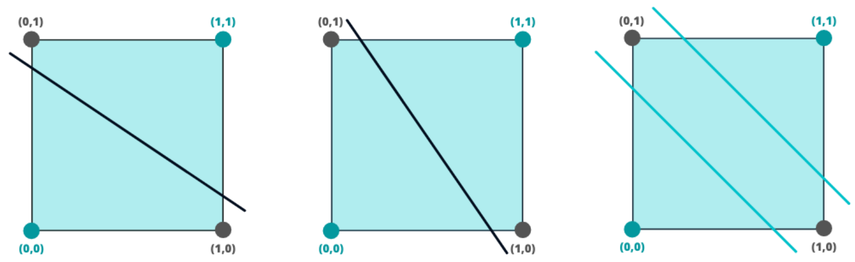
A typical example for the use of a Neural Network is solving the **XOR problem**. This blog article explains the **XOR problem** and how it can be solved by using a Neural Network. We will see that the problem can't be solved by a simple Perceptron. Instead, we need a Neural Network with a at least one hidden layer to solve the problem.

Logical XOR

The most popular truth tables are **OR** and **AND**. These tasks can be solved by a simple Perceptron. **XOR** stands for 'exclusive or'. The output of the XOR function has only a true value if the two inputs are different. If the two inputs are identical, the XOR function returns a false value. The following table shows the inputs and outputs of the XOR function.

Inputs		Outputs
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

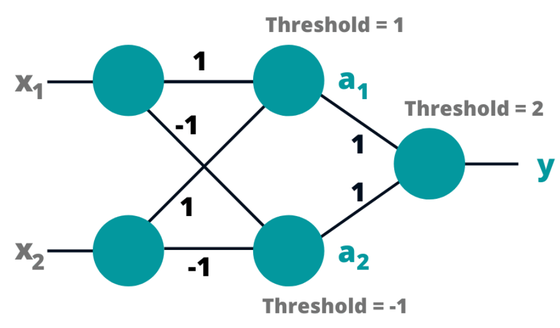
So, what is the special thing about the XOR function? In contrast to the **OR problem** and **AND problem**, the **XOR problem** cannot be linearly separated by a single boundaryline. Let's have a look at the following images:



The image on the right shows what the separation for the XOR function should look like. We can see that two boundarylines are needed to solve the problem.

Defining a Neural Network

The next step is to define a Neural Network that is able to solve the **XOR problem**. As mentioned above, the XOR function cannot be linearly separated by one boundaryline. This is also the reason why the function cannot be solved by a simple Perceptron. We need a more complex model. With the correct choice of functions and weight parameters, a Neural Network with one hidden layer is able to solve the **XOR problem**. For this, let's define the Neural Network we need.



In our model, the activation function is a simple threshold function. If a certain threshold value is exceeded, the function returns output 1, otherwise 0. So, the threshold value indicates from which value a neuron "fires". In addition to the threshold values, we also need to define the weight parameters.

So why did we choose these specific weights and threshold values for the Network? Let's have a look what happens in each layer.

Hidden Unit 1

$$a_1 = \begin{cases} 0 & x_1 + x_2 < 1 \\ 1 & x_1 + x_2 \geq 1 \end{cases}$$

Hidden Unit 2

$$a_2 = \begin{cases} 0 & -x_1 - x_2 < -1 \\ 1 & -x_1 - x_2 \geq -1 \end{cases}$$

Output Unit

$$y = \begin{cases} 0 & a_1 + a_2 < 2 \\ 1 & a_1 + a_2 \geq 2 \end{cases}$$

Calculations

Let's check if we really get the outputs of the **XOR-problem** with these formulas.

Case 1

The first case has the inputs $\mathbf{x_1 = 0}$ and $\mathbf{x_2 = 0}$ and the output should be **y = 0**.

$$\begin{aligned} &\text{Hidden Unit 1:} && x_1 + x_2 = 0 + 0 = 0 && \Rightarrow a_1 = 0 \\ &\text{Hidden Unit 2:} && -x_1 - x_2 = 0 - 0 = 0 && \Rightarrow a_2 = 1 \\ &\text{Output Unit:} && a_1 + a_2 = 0 + 1 = 1 && \Rightarrow y = 0 \end{aligned}$$

Case 2

The first case has the inputs $\mathbf{x_1 = 0}$ and $\mathbf{x_2 = 1}$ and the output should be **y = 1**.

$$\text{Output Unit:} \quad a_1 + a_2 = 1 + 1 = 2 \quad \Rightarrow y = 1$$

Case 3

The first case has the inputs $\mathbf{x_1 = 1}$ and $\mathbf{x_2 = 0}$ and the output should be $\mathbf{y = 1}$.

$$\text{Hidden Unit 1:} \quad x_1 + x_2 = 1 + 0 = 1 \quad \Rightarrow a_1 = 1$$

$$\text{Hidden Unit 2:} \quad -x_1 - x_2 = -1 - 0 = -1 \quad \Rightarrow a_2 = 1$$

$$\text{Output Unit:} \quad a_1 + a_2 = 1 + 1 = 2 \quad \Rightarrow y = 1$$

Case 4

The first case has the inputs $\mathbf{x_1 = 1}$ and $\mathbf{x_2 = 1}$ and the output should be $\mathbf{y = 0}$.

$$\text{Hidden Unit 1:} \quad x_1 + x_2 = 1 + 1 = 2 \quad \Rightarrow a_1 = 1$$

$$\text{Hidden Unit 2:} \quad -x_1 - x_2 = -1 - 1 = -2 \quad \Rightarrow a_2 = 0$$

$$\text{Output Unit:} \quad a_1 + a_2 = 1 + 0 = 1 \quad \Rightarrow y = 0$$

As you can see, the Neural Network generates the desired outputs.

Conclusion

Keep in mind that the XOR function can't be solved by a simple Perceptron. We need a Neural Network with as least one hidden layer. In this article you saw how such a Neural Network could look like. If you want dive deeper into Deep Learning and Neural Networks, have a look at our **Recommendations**.